Fragrantica Insights — Documento Técnico

Oct 30, 2026 · @Rebeca

Autora	Rebeca Dieb Holanda Silva
Matrícula	2519094
Instituição	Universidade de Fortaleza (Unifor)
Curso	Pós-Graduação em Engenharia de Dados, turma 2
Trabalho	Projeto de conclusão de curso

Introdução

Este é o projeto de conclusão de Rebeca Dieb Holanda Silva na Pós-Graduação em Engenharia de Dados da Universidade de Fortaleza (Unifor), turma 2. Ele reúne, num sistema único e funcional, os conhecimentos do curso (coleta e ingestão de dados, streaming de eventos, modelagem e armazenamento, qualidade e governança de dados e IA generativa) para resolver um problema real: decidir bem quando há opções demais.

O problema: decidir em meio ao excesso de opções

Mais opções nem sempre levam a escolhas melhores. No estudo clássico de Iyengar e Lepper (2000), uma banca com 24 sabores de geleia atraiu 60% dos clientes que passavam, contra 40% da banca com 6 sabores. Mas só 3% dos que pararam na banca maior compraram, contra quase 30% na menor. O fenômeno ficou conhecido como sobrecarga de escolha e foi popularizado por Schwartz (2004), para quem opções demais, muitas delas pouco diferentes entre si, sobrecarregam as pessoas e prejudicam o bem-estar em vez de aumentá-lo.

O efeito não aparece sempre. A meta-análise de Chernev, Böckenholt e Goodman (2015), com 99 observações e 7.202 participantes, identificou quatro fatores que tornam a sobrecarga mais provável: um conjunto de opções complexo, uma tarefa de decisão difícil, incerteza sobre as próprias preferências e o objetivo de decidir com pouco esforço.

A compra de perfume pela internet reúne os quatro. Só as 60 marcas mais populares do Fragrantica somam cerca de 15 mil perfumes. Cada um se descreve por atributos difíceis de comparar (notas olfativas, acordes, concentração). E o consumidor não pode sentir o cheiro antes de comprar, o que aumenta a incerteza sobre o que vai gostar e sobre como o perfume se comporta no uso real: quanto dura, quanto projeta, em que ocasiões funciona.

As reviews ajudam, mas também sobrecarregam

A resposta natural a essa incerteza é a experiência de quem já comprou. Chevalier e Mayzlin, analisando 2.394 livros em duas grandes livrarias on-line, mostraram que as avaliações dos clientes têm efeito causal sobre as vendas, e que uma avaliação negativa reduz as vendas mais do que uma positiva as aumenta.

Só que as reviews criam um segundo nível de excesso. Um perfume popular tem centenas de textos livres, em que comentários sobre fixação, preço e elogios se misturam a opiniões sobre o cheiro. A nota média do site resume tudo num número só e não diz se o perfume dura, se projeta ou para quem serve. O consumidor troca uma escolha difícil entre muitos produtos por uma leitura cansativa de muitas opiniões.

A abordagem: análise de sentimento por aspecto

A área que trata esse problema é a análise de sentimento baseada em aspectos. Hu e Liu (2004) propuseram resumir reviews por característica do produto: identificar as características comentadas, encontrar as frases de opinião sobre cada uma, classificá-las como positivas ou negativas e contar (resumo do método). A tarefa foi formalizada como desafio de avaliação por Pontiki et al. (2014), no SemEval-2014.

Os métodos clássicos dependem de regras ou de dados rotulados para treino, que não existem para reviews de perfume em português. Os grandes modelos de linguagem fazem essa leitura sem treino específico, mas trazem um risco novo: escrever com segurança o que as reviews não dizem. O desenho deste projeto responde a esse risco: o modelo só aponta quais frases falam de qual aspecto, e o código conta, calcula as notas de referência e mostra a frase real que sustenta cada afirmação.

Objetivos

Objetivo geral: reduzir a sobrecarga na escolha de um perfume, transformando as reviews de consumidores em um resumo por aspecto, confiável e rastreável, com o perfil de cliente para quem o perfume é indicado.

Objetivos específicos:

1. Coletar diariamente, e de forma incremental, as reviews do Fragrantica, e qualquer perfume sob demanda.
2. Levar os dados ao banco por streaming (Kafka), sem perdas nem duplicações.
3. Tratar e organizar os dados com regras de qualidade e de privacidade (limpeza, deduplicação, pseudonimização).
4. Dar nota a sete dimensões de uso com IA generativa, com a confiança de cada nota e as evidências que a sustentam.

5. Entregar tudo numa aplicação em que o consumidor escolhe qualquer perfume do site e recebe a análise.
6. Definir como medir a qualidade da análise contra rótulos humanos.

As dimensões respondem diretamente aos fatores de Chernev et al.: sete aspectos com a mesma escala tornam os perfumes comparáveis (menos complexidade), a experiência relatada por centenas de usuários reduz a incerteza sobre o uso real, e o resumo em uma tela atende a quem quer decidir com pouco esforço.

Organização do documento

As seções 1 e 2 dão a visão geral e a arquitetura. As seções 3 a 5 tratam da engenharia de dados: coleta, streaming, modelo de dados e ETL. As seções 6 a 8 tratam da camada de IA: o pipeline de análise, a engenharia de prompt e os provedores de modelo. A seção 9 descreve a aplicação e a operação; a 10, os testes e a validação; a 11, os resultados medidos. A seção 12 traz os achados, as limitações e as melhorias futuras, e as referências fecham o documento.

1. Visão geral

O Fragrantica Insights transforma as reviews de consumidores do Fragrantica em um "raio-x de uso real" de cada perfume: uma nota de 0 a 10 em sete dimensões, com a confiança de cada nota, as frases das reviews que a sustentam e um perfil do cliente ideal. No teste final, com o Claude, o Good Girl (Carolina Herrera) foi analisado a partir de 200 reviews, com confiança alta nas sete dimensões e 100% das menções apontando frases que existem nas reviews.

Problema. Um perfume popular tem centenas de reviews em texto livre. O consumidor quer saber quanto ele dura, se projeta, para quem serve e se vale o preço, mas essa informação está espalhada e misturada a opiniões sobre o cheiro. A nota média do site não separa esses aspectos.

Objetivo. Um projeto de ponta a ponta, em repositório, que:

1. coleta as reviews diariamente e de forma incremental, gravando-as num banco de dados por meio do Kafka;
2. permite escolher qualquer perfume do Fragrantica numa aplicação Streamlit, pelo catálogo ou colando o link;
3. usa IA generativa para dar nota a dimensões de uso (fixação, projeção, cheiro, versatilidade, originalidade, custo-benefício e elogios) e descrever o perfil de cliente ideal.

Escopo entregue.

Parte	O que foi feito
Coleta	Scraper com navegador real (SeleniumBase em modo UC) que passa pelo Cloudflare, coleta só as reviews novas e atende pedidos do app na hora
Streaming	Kafka com 5 tópicos, produtor idempotente, consumidor com gravação idempotente e fila de mensagens rejeitadas (DLQ)
Armazenamento	PostgreSQL com produtos, reviews, análises, pedidos de coleta e log de ingestão
Análise	Pipeline em duas etapas (extração menção a menção e síntese), com contagem e confiança calculadas pelo código e evidências que não podem ser inventadas
Aplicação	Streamlit com seleção por catálogo ou link, coleta sob demanda, análise, reviews e painel do pipeline; modo leitura sem chave de API
Operação	Analista automático em segundo plano, carga em lote, cópia dos dados no repositório para avaliação
Qualidade	72 testes automatizados e um protocolo de validação contra rótulos humanos (precisão, recall, kappa de Cohen)

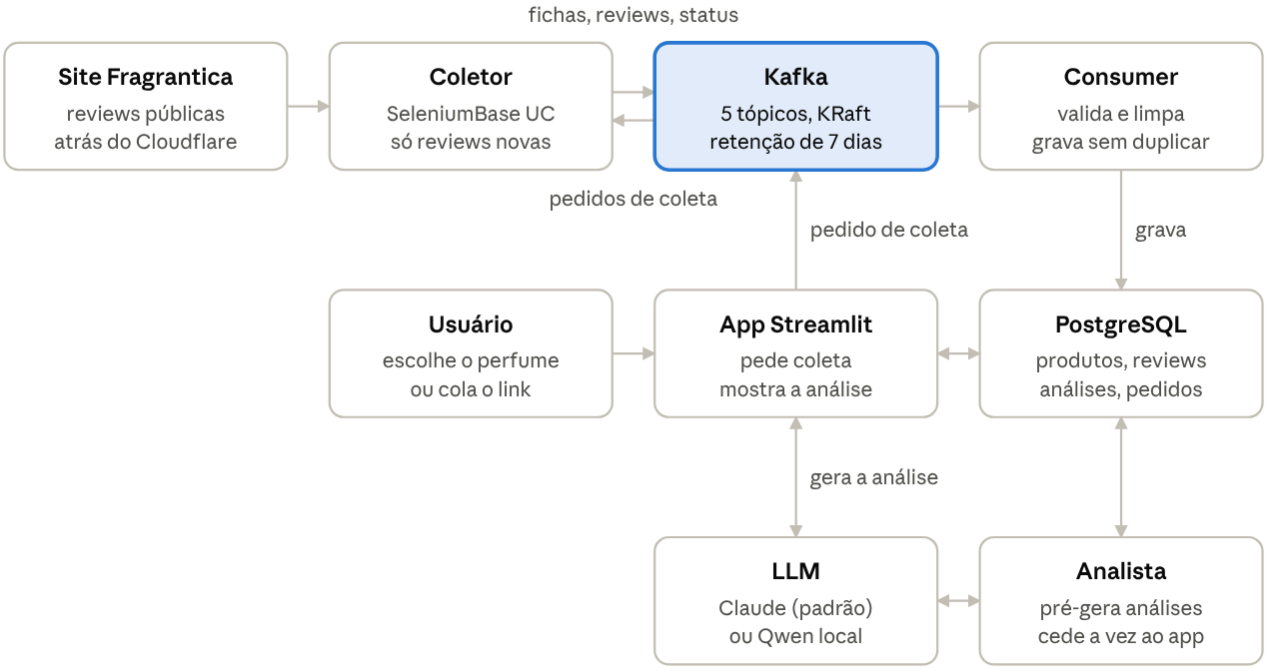
Stack. Python 3.12, SeleniumBase, Apache Kafka 3.8 (KRaft), PostgreSQL 16, Pydantic, Streamlit, Docker Compose. LLM: Claude (Haiku 4.5 na extração e Sonnet 5 na síntese) por padrão; Qwen 2.5 3B via Ollama como alternativa local e gratuita.

Este documento registra as decisões de cada etapa e o motivo de cada uma. O passo a passo de instalação e uso está no README do repositório.

2. Arquitetura

O sistema tem seis serviços em Docker Compose, e a coleta é desacoplada de todo o resto pelo Kafka: o coletor só publica eventos, o consumer só grava, e o app só lê o banco e publica pedidos.

A coleta e o app só se falam pelo Kafka; tudo fica no PostgreSQL



arquitetura · 8 componentes, Kafka no centro

As setas mostram quem chama quem. O coletor também consulta o banco para saber o que coletar e qual a review mais recente de cada perfume (seta omitida para não cruzar o desenho).

Serviço	Papel	Por que existe separado
`kafka`	Barramento de eventos (modo KRaft, sem ZooKeeper)	Desacopla a coleta, lenta e sujeita a bloqueio, da gravação e do app
`postgres`	Banco de registro	Consultas relacionais por perfume, unicidade e transações para a gravação idempotente
`coletor`	Scraper: atende pedidos do app e faz a rodada diária	Precisa de Chrome e de uma tela virtual; imagem própria (`Dockerfile.coleta`)
`consumer`	Valida, limpa, deduplica e grava em lote	Único ponto de escrita no banco: regras de ETL num lugar só
`app`	Interface Streamlit	Só lê o banco e publica pedidos; nunca coleta direto
`analista`	Gera análises em segundo plano	Tira a espera do usuário sem competir com ele pelo modelo

Decisões de arquitetura.

- **Kafka entre coleta e banco**, e não gravação direta pelo scraper. Se o banco cair, as mensagens esperam no tópico (retenção de 7 dias) e o consumer retoma do último offset confirmado. Outra fonte (outro site, o notebook do Colab) entra publicando no mesmo contrato, sem mexer no consumer.
- **O app pede coleta pelo Kafka**, em vez de chamar o scraper. O coletor mantém um único navegador aberto e atende os pedidos entre um perfume e outro da rodada diária, sem abrir um Chrome por usuário.
- **Um broker e três partições.** Suficiente para o volume (milhares de reviews por dia) e para mostrar o paralelismo por chave; replicação fica fora do escopo de uma máquina só.
- **PostgreSQL, e não um banco de documentos.** Os dados são relacionais (perfume → reviews → análises), e a idempotência depende de restrições de unicidade. O resultado da análise, que tem forma variável, fica numa coluna JSONB.
- **Tudo em Docker Compose**, com `make` como atalho. Um comando sobe o projeto em qualquer máquina; o mesmo arquivo serve para uma VM.

3. Coleta de dados

A coleta usa um navegador real controlado por código, porque o Fragrantica fica atrás do Cloudflare e carrega as reviews por JavaScript. Ela começou num notebook do Google Colab e foi levada para o serviço `coletor`, mantendo as mesmas soluções.

3.1 Obstáculos e decisões

Obstáculo encontrado	Decisão
O `chromium-browser` do Colab era só um atalho, sem navegador	Instalar o Google Chrome pelo pacote `.deb` oficial
Páginas que não terminavam de carregar (anúncios, rastreadores) e estouravam o tempo	Carregamento `eager` (não espera imagens e scripts de terceiros), `window.stop()` após o conteúdo principal e imagens bloqueadas
Chrome sem tela (*headless*) bloqueado pelo Cloudflare	SeleniumBase em modo UC (*undetected*) com o navegador visível numa tela virtual (Xvfb, via `pyvirtualdisplay`); até 3 tentativas por página

Obstáculo encontrado	Decisão
A primeira versão encontrava 0 reviews	Inspeção do HTML: cada review é um `div` com `id="review_<número>"`; as reviews só carregam quando a seção fica visível, então o coletor rola a página aos poucos até ela
Só parte das reviews aparece; o resto vem pelo botão "mais resenhas"	Cliques repetidos no botão, até `MAX_CLIQUES_REVIEWS` (100), o que cobriu perfumes com mais de 200 reviews
Datas recentes relativas ("a day ago", "há um mês") e antigas no formato MM/DD/YY	Texto original guardado e data interpretada para os dois formatos, em inglês e português
Autor em lugares diferentes do HTML	Leitura com alternativas; nome pseudonimizado na gravação (seção 5)
Lentidão ao procurar elementos um a um	Um único JavaScript por página extrai ficha e reviews de uma vez (`scraper/js.py`)
Memória crescendo em coletas longas	O Chrome é reiniciado a cada 150 páginas

3.2 O que é coletado

- **Ficha do perfume:** marca, nome, gênero, ano, nota média e votos do site, número de reviews informado pelo site, perfumistas, acordes e pirâmide olfativa (notas de topo, coração e base).
- **Reviews:** id do site (número do `div`, que cresce com o tempo), autor, data e texto.
- **Catálogo:** as 60 marcas da seção "Marcas Mais Populares" e os links dos perfumes de cada uma (cerca de 15 mil perfumes, em ~26 s por marca). O id do perfume vem do fim da URL, e todos os links são normalizados para `fragrantica.com.br`, então links de qualquer idioma do site apontam para o mesmo perfume.

3.3 Coleta incremental

O `review_id` do site cresce com o tempo, então o maior id já gravado de cada perfume funciona como marca d'água. O coletor para de clicar em "mais resenhas" assim que a página mostra um id menor ou igual a essa marca, e publica só as reviews novas. Um perfume revisitado no dia seguinte custa uma página e poucos segundos.

Em cada visita, as reviews são publicadas **antes** da ficha. Quando a ficha marca a visita no banco (`ultima_coleta`), as reviews já estão no tópico; assim o app nunca vê um perfume "coletado" sem as reviews.

3.4 Dois modos de coleta (fluxo híbrido)

Modo	Quando	O que faz
Rodada diária	`HORARIO_COLETA` (03:00, fuso de Fortaleza)	Monta o catálogo se o banco estiver vazio ou a cada 7 dias; depois visita até 200 perfumes, os coletados há mais tempo primeiro
Sob demanda	Quando o usuário pede a análise de um perfume sem reviews ou coletado há mais de 7 dias	O app publica um pedido; o coletor atende entre um perfume e outro da rodada e publica o andamento (`coletando`, `concluido`, `erro`)

O fluxo híbrido foi escolhido contra as duas alternativas puras. Só coleta diária limitaria o app aos perfumes do catálogo e deixaria os demais desatualizados. Só sob demanda faria todo usuário esperar a coleta. No híbrido, os populares respondem na hora e qualquer outro perfume do site funciona com uma espera de 1 a 2 minutos.

Decisões da coleta sob demanda:

- **Limite de reviews:** coleta só as `LLM_MAX_REVIEWS` (100) mais recentes, que são as que a análise usa. Um perfume com 2 mil reviews não faz o usuário esperar 20 minutos.
- **Validade de 7 dias** (`VALIDADE_COLETA_DIAS`): reviews coletadas há menos tempo são usadas direto; o botão "Atualizar reviews" força uma coleta.
- **Pedidos expiram em 10 minutos** (`ESPERA_COLETA_S`): um pedido feito com o coletor fora do ar não é atendido horas depois, quando ninguém mais espera por ele.
- **Como o app sabe que acabou:** o pedido está concluído quando o perfume tem `ultima_coleta` posterior ao pedido e o maior `review_id` gravado alcança o informado no status. As duas condições são necessárias porque status e reviews chegam por tópicos diferentes.

Uso responsável. Pausa de 1,5 s entre perfumes, um navegador só, apenas páginas públicas e autores pseudonimizados.

4. Streaming com Kafka

O Kafka garante que nenhuma review coletada se perca e que nenhuma seja gravada duas vezes. Isso vem da combinação de entrega pelo menos uma vez no Kafka com gravação idempotente no banco, o que dá o efeito de "exatamente uma vez" no resultado final.

4.1 Tópicos

Tópico	Chave	Conteúdo	Quem publica → quem lê
`fragrantica.produtos`	id do perfume	Ficha (`tipo=detalhes`) ou entrada de catálogo (`tipo=catalogo`)	coletor, carga CSV → consumer
`fragrantica.reviews`	id do perfume	Uma review: id, autor, data, texto	coletor, carga CSV → consumer
`fragrantica.pedidos`	id do perfume	Pedido de coleta sob demanda	app → coletor e consumer
`fragrantica.pedidos.status`	id do perfume	Andamento: `na_fila` , `coletando` , `concluido` , `erro`	coletor → consumer
`fragrantica.dlq`	chave original	Mensagem rejeitada + motivo	consumer → inspeção manual

Todos têm 3 partições e retenção de 168 horas (7 dias). O consumer cria os tópicos ao iniciar, se ainda não existirem.

4.2 Decisões

- **Chave = id do perfume.** Todas as mensagens de um perfume vão para a mesma partição e mantêm a ordem entre si; perfumes diferentes podem ser processados em paralelo.
- **Contratos validados com Pydantic, com `versao_schema` .** Os quatro eventos (`ProdutoEvento` , `ReviewEvento` , `PedidoColeta` , `StatusPedido`) são modelos Pydantic usados por quem publica e por quem lê. Um campo novo no futuro vem com uma versão nova, e o consumer pode tratar as duas.
- **Produtor idempotente:** `acks=all` e `enable.idempotence=true` . Uma nova tentativa de envio, depois de uma falha de rede, não duplica a mensagem no broker.
- **Commit manual depois do banco.** O consumer lê lotes de até 500 mensagens (ou o que chegar em 2 s), grava tudo numa transação e só então confirma o offset. Se o processo cair no meio, o lote é relido e regravado sem duplicar, porque toda gravação usa `ON CONFLICT` .

- **Fila de rejeitadas (DLQ).** Mensagem que não passa na validação vai para `fragrantica.dlq` com o tópico de origem e o motivo, e o lote segue. Uma mensagem ruim não trava o pipeline nem se perde em silêncio.
- **Ordem dentro do lote:** fichas primeiro, depois reviews, depois pedidos e status, e por último o log de ingestão. O app só lê o andamento de um pedido depois do commit que gravou as reviews dele.

4.3 Mensagens fora de ordem

Os tópicos são independentes, então duas situações aparecem na prática e foram tratadas:

- **Review antes da ficha do perfume.** Como `reviews.id_perfume` tem chave estrangeira para `produtos`, o consumer cria antes um registro mínimo do perfume (marca, nome e link do próprio evento), completado quando a ficha chega. Esse caso apareceu como erro de chave estrangeira no primeiro teste e motivou a regra.
- **Status antes do pedido, ou `coletando` depois de `concluido`.** A tabela `pedidos_coleta` só deixa o status avançar (`na_fila` → `coletando` → `concluido` ou `erro`). Um status atrasado não faz um pedido concluído "voltar" para coletando.

Os dois casos têm teste automatizado, inclusive contra um PostgreSQL real.

5. Modelo de dados e ETL

O banco guarda o texto original de cada review ao lado do texto limpo, e toda gravação é idempotente. Assim dá para reprocessar qualquer coisa, e auditar a limpeza, sem perder nem duplicar dados.

5.1 Tabelas

Tabela	Uma linha por	Campos principais e decisões
`produtos`	perfume	Id do site como chave; ficha completa (acordes e notas em arrays); `fonte` (`scraper`, `csv`, `exemplo`); `ultima_coleta`. A ficha nova sobrescreve, mas com `COALESCE`: um campo vazio na coleta nova não apaga um já preenchido

Tabela	Uma linha por	Campos principais e decisões
`reviews`	review	Id do site como chave; `texto_original` e `texto_limpo`; data como texto e interpretada; `n_palavras`; `informativa`; `hash_conteudo` único; autor pseudonimizado
`analises`	análise gerada	Chave única: perfume + provedor + modelo da síntese + versão do prompt + assinatura das reviews usadas; resultado em JSONB; % de menções válidas; duração
`pedidos_coleta`	pedido sob demanda	Status que só avança, reviews coletadas, maior id coletado, mensagem de erro
`uso_llm`	uso do modelo em andamento	Quem está usando o LLM agora (`app` ou `analista`); é como o analista sabe ceder a vez
`ingestao_log`	lote do consumer	Mensagens, produtos, reviews novas, duplicadas e rejeitadas
`vw_produtos` (view)	perfume	Produto + nº de reviews, nº de informativas, review mais recente e maior `review_id`

Os scripts de criação ficam em `sql/` e são idempotentes (`CREATE ... IF NOT EXISTS`). O consumer e o analista aplicam todos ao iniciar, então um banco criado por uma versão anterior ganha as tabelas novas sem perder dados.

5.2 Regras do ETL

1. **Limpeza:** normaliza a codificação (Unicode NFC), desfaz entidades HTML, remove tags e textos de interface que vazam na coleta, e junta espaços e linhas em branco repetidos. O texto original fica guardado sem alteração.
2. **Review informativa:** 5 palavras ou mais (`MIN_PALAVRAS_INFORMATIVA`). Reviews como "amei!!" ficam no banco, mas a análise prioriza as informativas e só é liberada com pelo menos 5 delas (`MIN_REVIEWS_ANALISE`).
3. **Deduplicação em duas camadas:** pelo id da review (a mesma review coletada duas vezes) e por um hash SHA-256 de perfume + texto normalizado (o mesmo texto publicado com ids diferentes, como uma review repostada).

4. **Pseudonimização:** o nome do autor vira `avaliador_` + 10 caracteres de um hash SHA-256. O mesmo autor recebe sempre o mesmo apelido, então dá para agrupar reviews de uma pessoa sem guardar o nome. Pode ser desligada (`ANONIMIZAR_AUTORES=false`).
5. **Datas:** a data relativa ("a day ago", "há 3 meses") é convertida a partir do momento da coleta; o texto original fica em `data_texto`, porque a conversão de "há um mês" é aproximada.

5.3 Por que guardar o resultado da análise no banco

Cada análise custa tempo (minutos com um modelo local) ou dinheiro (centavos com a API). Gerar a cada visita seria caro e daria resultados diferentes para o mesmo perfume. A chave da tabela `analises` define quando uma análise ainda vale:

- **assinatura das reviews:** um hash dos ids das reviews lidas. Reviews novas mudam a assinatura, e o app avisa que dá para atualizar;
- **provedor e modelo:** análises de modelos diferentes não se misturam, o que permite compará-las;
- **versão do prompt e modo:** um prompt novo invalida as análises antigas sem apagá-las.

6. Camada de GenAI

O modelo de linguagem faz só o que exige leitura: apontar quais frases falam de qual aspecto e escrever o texto final. Contar, calcular a nota de referência, medir a confiança e conferir as evidências são tarefas do código. Essa divisão é a principal decisão da camada de IA: o que é número sai de uma conta verificável, e o que é texto sai do modelo, preso a evidências reais.

6.1 Taxonomia

Sete dimensões fixas, definidas a partir do que os consumidores de perfume discutem e do que o enunciado pedia (durabilidade, projeção, fixação). "Durabilidade" e "fixação" são tratadas como a mesma coisa, porque nas reviews os dois termos descrevem quanto tempo o perfume dura.

Chave	Dimensão	Definição usada no prompt
`fixacao`	Fixação / durabilidade	Quanto tempo o perfume permanece perceptível na pele ou na roupa
`projecao`	Projeção e rastro	O quanto o cheiro se espalha e é percebido por outras pessoas (sillage)

Chave	Dimensão	Definição usada no prompt
`cheiro`	Qualidade do cheiro	O quanto o cheiro em si agrada: beleza, equilíbrio, sensação de qualidade
`versatilidade`	Versatilidade	Em quantas situações funciona: clima, horário, ocasiões, trabalho
`originalidade`	Originalidade	O quanto é único versus genérico ou parecido com outros perfumes
`custo_beneficio`	Custo-benefício	Relação entre preço, rendimento e o que o perfume entrega
`elogios`	Elogios recebidos	Reação de outras pessoas a quem usa: elogios, comentários, indiferença

O conjunto é **fechado**: na saída do modelo, a dimensão é um enum com essas sete chaves. Isso torna as análises comparáveis entre perfumes e impede que o modelo crie dimensões. O perfil de cliente ideal (para quem é, para quem não é, ocasiões, estações, período) não é uma dimensão com nota: é um texto da síntese, apoiado nas ocasiões e perfis citados nas reviews.

6.2 Pipeline da análise (modo detalhado)

1. **Seleção:** até 100 reviews (`LLM_MAX_REVIEWS` ; 40 com o modelo local), informativas primeiro e, entre elas, as mais recentes. Perfumes são reformulados com o tempo, e as reviews recentes descrevem o que se compra hoje. Cada review é cortada em 1.500 caracteres.
2. **Frases numeradas e filtro:** o código divide cada review em frases e numera (`[3.2]` = review 3, frase 2). Frases sem nenhuma pista de dimensão (história da compra, saudações) não são enviadas (`FILTRAR_FRASES`). As pistas são largas de propósito: na dúvida, a frase vai.
3. **Extração em lotes** (40 reviews por chamada no Claude, 4 em paralelo; 10 por chamada no modelo local): para cada menção, o modelo devolve review, frase, dimensão e sinal (`+` , `-` , `0`).
4. **Conferência:** menção que aponta uma review ou frase inexistente é descartada e contada. A evidência guardada é a frase real, recortada pelo código.
5. **Agregação pelo código:** por dimensão, quantas reviews elogiam, criticam ou são neutras (cada review conta uma vez por dimensão e sinal), o índice de 0 a 10, a confiança e até 8 frases de evidência sem repetição, alternando críticas e elogios.
6. **Trava contra texto inventado:** sem nenhuma menção válida, a análise para aqui e não é salva. Uma síntese escrita sobre nada seria inventada.

7. **Síntese:** o modelo recebe a ficha técnica, as contagens, o índice e até 6 frases por dimensão, e devolve a nota e o resumo de cada dimensão, pontos fortes e fracos, perfil de cliente ideal e resumo geral.
8. **Ajustes finais pelo código:** nota ancorada no índice (6.3); pontos fortes, fracos e ocasiões preenchidos pelas contagens quando o modelo os deixa vazios.
9. **Cache:** o resultado vai para a tabela `analises` (seção 5.3).

6.3 Índice, nota e confiança

O índice é a referência estatística de cada dimensão. Elogio vale 1, menção neutra vale 0,5 e crítica vale 0, com uma suavização de k = 2 relatos neutros imaginários:

$$\text{índice} = 10 \times \frac{P + 0{,}5\,N_0 + 0{,}5\,k}{P + N + N_0 + k}, \qquad k = 2$$

P, N e N0 são as reviews com menção positiva, negativa e neutra. A suavização puxa para o meio as dimensões com poucos relatos e quase não mexe nas demais: 3 críticas dão 2,0 (e não 0), 40 elogios dão 9,8. Ela foi adicionada depois que a originalidade do Good Girl saiu 0,0 a partir de poucas críticas.

Situação da dimensão	Nota exibida	Motivo
Nenhuma menção	sem nota	Não há base; o resumo diz que as reviews não falam disso
1 a 4 menções	o índice	Poucos relatos: o modelo exagera com facilidade
5 ou mais	a do modelo, limitada a 2 pontos do índice	O modelo pesa a intensidade das frases ("dura 12 horas" vale mais que "dura bem"), mas não pode contradizer as contagens
Modelo não deu nota	o índice	Modelos pequenos às vezes devolvem a nota vazia

A tela diz quando a nota veio do índice ou foi ajustada.

A **confiança** depende só de quantas reviews comentam a dimensão: alta com 15 ou mais, média com 5 a 14, baixa com 1 a 4. Ela não é pedida ao modelo, porque modelos de linguagem são mal calibrados quando avaliam a própria certeza.

6.4 Medidas contra alucinação

Risco	Medida
Dimensão inventada	Enum fechado no esquema de saída
Trecho que não está na review	O modelo aponta o número da frase; o código recorta a frase real
Review ou frase inexistente	Descartada e contada ("N menções descartadas"); % de menções válidas no rodapé
Números inventados ("a maioria relata 8 horas")	Contagens, índice e confiança vêm do código; o prompt proíbe números que não estejam no material
Texto confiante sem base	Análise sem menções válidas não é salva
Nota exagerada	Ancorada no índice suavizado
Leitor que confia demais	Aviso de conteúdo gerado por IA com modelo, data, nº de reviews e % de menções válidas; frases de evidência em cada dimensão

6.5 Saída estruturada

Os contratos de saída são modelos Pydantic (`ExtracaoLote`, `SinteseProduto`, `AnaliseRapida`). O esquema JSON gerado a partir deles é imposto ao modelo: no Claude, como o esquema de uma ferramenta de uso obrigatório (*tool use* com `tool_choice` forçado); no Ollama, pelo parâmetro `format`. A resposta é validada pelo Pydantic e, se vier fora do contrato, volta ao modelo uma vez com o erro de validação.

7. Engenharia de prompt

O prompt passou por cinco versões, e cada uma respondeu a uma falha observada num teste real, não a uma suposição. A lição central: com modelos pequenos, o formato da resposta pesa mais que a redação das instruções, e tudo o que o modelo puder copiar das instruções ele copia.

7.1 Princípios usados em todas as versões

- **Papel e objetivo no início** ("analista de reviews de perfumes", "raio-x de uso real... para decidir a compra"), para o tom e o critério de relevância.

- **Definição explícita de cada dimensão**, gerada da taxonomia no código: o prompt e o enum de saída nunca divergem.
- **Regras numeradas e curtas**, com a regra de omissão explícita ("Na dúvida, deixe de fora"): é melhor perder uma menção fraca que inventar uma.
- **Um exemplo resolvido** de entrada e saída, que mostra o formato e casos difíceis (uma frase com duas dimensões; comparação com outro perfume como originalidade negativa).
- **Dados entre marcadores** (`<reviews>` , `<material>`), separando o que é instrução do que é conteúdo do usuário. Uma review que diga "ignore as instruções" continua sendo só texto a classificar.
- **Saída imposta por esquema** (seção 6.5), em vez de pedir JSON em texto livre.
- **Versão no código** (`VERSAO_PROMPT`): mudou o texto, muda a versão, e o cache não mistura análises de versões diferentes.

7.2 Evolução das versões

Versão	Formato pedido ao modelo	Falha que motivou a próxima versão
v1	Por review: lista de atributos (dimensão, polaridade, trecho literal), com o id longo da review do site	Com o Qwen 2.5 3B, as 15 reviews do Wind Flowers voltaram sem nenhuma menção (418 tokens, tudo vazio), embora falassem de fixação, projeção, preço e elogios. Listas que o próprio modelo decide preencher são fáceis de deixar vazias
v2	Formato plano: reviews numeradas de 1 a N; para cada review e cada uma das 7 dimensões, uma opção fechada (`positiva` , `negativa` , `neutra` , `nao_menciona`) e o trecho; exemplo resolvido	O modelo passou a preencher, mas 4 de 5 evidências de fixação eram "dura o dia todo", frase que não estava nas reviews: ele copiou dos exemplos da taxonomia. A confiança saiu alta com contagens infladas, a nota veio vazia e o texto teve erros ("afixação")

Versão	Formato pedido ao modelo	Falha que motivou a próxima versão
v3	Igual à v2, mas a descrição das dimensões na extração perdeu as frases de exemplo (ficaram só palavras-chave), e menção com trecho que não está na review passou a ser descartada das contagens	A análise ficou honesta, mas lenta: ~185 tokens de saída por review, pois o formato plano escreve as 7 dimensões de toda review, inclusive as não mencionadas. A ~8 tokens/s, 40 reviews levavam cerca de 16 minutos
v4	Evidência pelo número da frase: o código numera as frases (`[3.2]`) e o modelo devolve só as menções existentes, com chaves curtas (`{"r":3,"f":2,"d":"fixacao","p":"+"}`); síntese mais curta; modo rápido de uma chamada (seção 8)	Detalhado: 2 a 3,5 min para 40 reviews, 98% de menções válidas. Restou a originalidade do Good Girl com nota 0,0 a partir de poucas críticas, e quase nenhuma dimensão chegando à confiança alta
v5	Mesmo prompt; índice suavizado e nota do modelo ancorada nele (seção 6.3); o analista passou a ler até 100 reviews	Versão atual

A v4 resolveu duas coisas de uma vez. A resposta ficou cerca de 4 vezes menor, porque o modelo escreve só o que existe e com chaves de uma letra. E a evidência deixou de poder ser inventada: o modelo aponta um número, e o texto mostrado é a frase real que o código recorta. Número que não existe na entrada é descartado e contado.

7.3 Prompt de extração (v5)

Prompt de sistema, como enviado ao modelo:

```
Você é um analista de reviews de perfumes. Encontre, em cada review, as
frases
que falam de alguma das dimensões abaixo.

Dimensões:
- fixacao (Fixação / durabilidade): Quanto tempo o perfume permanece
perceptível na pele ou na roupa.
- projecao (Projeção e rastro): O quanto o cheiro se espalha e é percebido
por outras pessoas (sillage).
- cheiro (Qualidade do cheiro): O quanto o cheiro em si agrada: beleza,
equilíbrio, sensação de qualidade.
- versatilidade (Versatilidade): Em quantas situações funciona: clima,
horário, ocasiões, ambiente de trabalho.
```

```
- originalidade (Originalidade): O quanto é único versus genérico ou parecido
com outros perfumes.
- custo_beneficio (Custo-benefício): Relação entre preço, rendimento e o que
o perfume entrega.
- elogios (Elogios recebidos): Reação de outras pessoas a quem usa: elogios,
comentários, indiferença.

As reviews vêm divididas em frases numeradas: [3.2] é a frase 2 da review 3.

Para cada menção, devolva um item em "mencoes" com:
- r: número da review; f: número da frase; d: a dimensão;
- p: "+" se elogia, "-" se critica, "0" se só descreve sem elogiar nem
criticar.

Regras:
1. Liste SÓ o que a review realmente diz. Dimensão não mencionada não entra.
Na dúvida, deixe de fora.
2. Uma frase pode gerar várias menções (dimensões diferentes).
3. Use apenas números de review e de frase que aparecem na entrada.
4. "perfis" e "ocasioes": expressões curtas, só quando as reviews disserem.
Pistas: comparação com outro perfume ou clone -> originalidade; preço e
rendimento ->
custo_beneficio; horas, duração, longevidade -> fixacao; alcance, rastro,
sillage -> projecao;
clima, horário, ocasiões -> versatilidade; reação de outras pessoas ->
elogios; o cheiro em si
(bonito, enjoativo, sintético) -> cheiro.

Exemplo
Entrada:
<review numero="1">
[1.1] Cheiro lindo de baunilha, mas some em 2 horas.
[1.2] Parecido demais com outro perfume da moda.
[1.3] Recebo elogios no trabalho.
</review>
Resposta: {"mencoes": [{"r": 1, "f": 1, "d": "cheiro", "p": "+"}, {"r": 1,
"f": 1, "d": "fixacao", "p": "-"}, {"r": 1, "f": 2, "d": "originalidade",
"p": "-"}, {"r": 1, "f": 3, "d": "elogios", "p": "+"}], "perfis": [],
"ocasioes": ["trabalho"]}
```

Mensagem do usuário, montada pelo código para cada lote:

```
Perfume: Good Girl (Carolina Herrera)

<reviews>
<review numero="1">
[1.1] ...
[1.3] ...
</review>
...
</reviews>

Liste as menções.
```

Decisões deste prompt:

- **Pistas por palavra-chave, não frases de exemplo.** Na v2, "dura o dia todo" estava nos exemplos e virou evidência inventada. Palavras soltas ("horas, duração, longevidade") orientam sem dar nada pronto para copiar.
- **O exemplo usa marca e perfume genéricos** ("outro perfume da moda"), para não ensinar nomes que o modelo repetiria.
- **Os números das frases enviadas preservam a numeração original.** Se o filtro tira a frase 2, a review mostra `[1.1]` e `[1.3]`; a frase continua identificável no texto completo.
- **O exemplo de saída mostra uma frase com duas menções** (cheiro positivo e fixação negativa na mesma frase), o caso que o 3B mais errava.

7.4 Prompt de síntese (v5)

```
Você escreve o "raio-x de uso real" de um perfume para ajudar um consumidor a
decidir a compra. Você recebe contagens agregadas de menções por dimensão e
trechos reais
de reviews. Baseie-se somente nesse material.

Dimensões:
[as 7 definições, aqui com exemplos, porque a síntese não copia evidências]

Regras para a NOTA (0 a 10) de cada dimensão:
- Devolva as 7 dimensões. Toda dimensão com 1 menção ou mais TEM nota
(número, nunca null).
- Reflita o equilíbrio entre menções positivas e negativas e a intensidade
dos trechos.
  O "indice_calculado" é uma referência estatística; afaste-se dele só com
motivo claro nos trechos.
- Dimensão com 0 menções: nota = null e resumo dizendo que as reviews não
falam disso.
```

```
- Com poucas menções (1 a 4), seja conservador e diga no resumo que há poucos
relatos.

Regras para o texto:
- Português do Brasil, direto, sem exageros de marketing.
- Resumo de cada dimensão: 1 frase curta, só sobre AQUELA dimensão (não
misture fixação com
  projeção), citando o que os consumidores relatam.
- Escreva frases simples e confira a ortografia de cada palavra.
- pontos_fortes e pontos_fracos: 2 a 4 itens curtos cada (pode ficar vazio só
se não houver).
- perfil_cliente.ocasioes, estacoes e periodo: preencha a partir de
"ocasioes_citadas" e dos trechos.
- perfil_cliente.descricao: 2 a 3 frases dizendo para quem o perfume é ideal
(gosto olfativo,
  estilo, ocasiões, intensidade desejada) e para quem não é.
- resumo_geral: 2 a 3 frases.
- Não invente notas olfativas, preços ou números que não estejam no material.
```

A mensagem do usuário é um JSON entre `<material>` e `</material>`, com a ficha técnica (nome, marca, gênero, ano, acordes e notas), o número de reviews analisadas e, por dimensão: menções, positivas, negativas, neutras, índice calculado e até 6 frases de evidência. Vão também os 15 perfis e as 15 ocasiões mais citados.

Decisões deste prompt:

- **A síntese não recebe as reviews**, só o material agregado. A entrada fica pequena e o modelo não tem como recontar à sua maneira.
- **O índice vai junto como âncora**, e o código ainda limita a nota final a 2 pontos dele (6.3). A instrução sozinha não bastou com o 3B, que deixava notas vazias ou extremas.
- **"Toda dimensão com menção TEM nota"** entrou na v3, depois de notas vazias com 15 menções.
- **"Só sobre AQUELA dimensão"** entrou depois que o resumo de fixação falou de projeção.
- **"Confira a ortografia"** ajuda pouco num modelo de 3B ("afixação" continuou aparecendo); o problema só sumiu com o Claude.
- **A ficha técnica entra para dar contexto ao perfil** (um gourmand doce tende a ser noturno), e a regra final proíbe usar dela notas ou preços que as reviews não sustentam.

7.5 Prompt do modo rápido

Uma chamada só com todas as reviews. Para cada dimensão, o modelo lista os números das reviews que elogiam, criticam ou são neutras, até 3 frases de evidência ("3.2"), a nota e o resumo; depois, o perfil, os pontos fortes e fracos e o resumo geral. No esquema, as listas vêm antes da nota, para o modelo apontar as reviews antes de opinar. O resultado deste modo está na seção 11.

8. Provedores de LLM e modos de análise

O projeto terminou usando o Claude pela API: Haiku 4.5 na extração e Sonnet 5 na síntese. O caminho passou por um modelo aberto rodando localmente, que ficou como alternativa gratuita; os limites dele explicam várias decisões da seção 7.

8.1 Abstração de provedor

Todo provedor implementa uma única operação: `gerar(modelo, sistema, usuario, schema, max_tokens)`, que devolve um objeto Pydantic já validado. O pipeline não sabe qual provedor está usando; trocar é mudar `LLM_PROVEDOR` no `.env`.

Provedor	Modelos	Como a saída estruturada é garantida	Limites por análise	Uso
`anthropic` (padrão)	Haiku 4.5 na extração, Sonnet 5 na síntese	Ferramenta de uso obrigatório com o esquema JSON	100 reviews, lotes de 40, 4 chamadas em paralelo	Produção
`ollama`	Qwen 2.5 3B (opcional: outro modelo só na síntese)	Parâmetro `format` com o esquema JSON	40 reviews, lotes de 10, uma chamada por vez	Alternativa local e gratuita
`demo`	Nenhum: classificação por palavras-chave	—	todas	Testes automatizados e demonstração sem IA

Modelos diferentes por etapa no Claude: a extração é a maior parte dos tokens e é uma tarefa de classificação, então vai para o modelo mais barato; a síntese é uma chamada só, e é o texto que o usuário lê, então vai para o mais capaz.

8.2 Do modelo local ao Claude

Etapa	Configuração	O que aconteceu	Decisão tomada
1	Claude	A conta exigia compra mínima de créditos	Testar um modelo aberto, gratuito
2	Qwen 2.5 7B no Ollama, contexto de 16 mil tokens	O modelo não coube na GPU do Mac (23% rodando na CPU) e a análise estourou o tempo	Contexto de 8 mil, lotes de 10 reviews, no máximo 40 reviews, teto de saída por chamada, tempo limite de 10 min com mensagem clara
3	Qwen 2.5 3B (2,3 GB, 100% na GPU)	Rodou, mas a extração voltou vazia, depois inventou trechos e foi lenta (~8 tokens/s)	Prompts v2 a v4 (seção 7)
4	3B na extração + 7B só na síntese	Os dois modelos e o Docker não couberam juntos na memória: a chamada passou de 10 min e o Streamlit caiu	Desistir do 7B nesta máquina
5	Claude (Haiku + Sonnet)	200 reviews, confiança alta nas 7 dimensões, 100% das menções válidas, texto coerente	Padrão do projeto

Detalhes de cada provedor que vieram desses testes:

- **Ollama:** o teto de saída (`num_predict`) é proporcional ao lote (120 tokens por review + 300), porque um modelo local às vezes "empaca" gerando espaços em branco e o teto impede que isso trave a análise. O modelo fica carregado por 30 min entre chamadas (`keep_alive`), temperatura 0,1, e o app avisa se o Ollama está fechado ou o modelo não foi baixado.
- **Claude:** piso de 8 mil tokens de saída (os tetos apertados são para o modelo local; na API eles só cortariam o JSON); até 6 novas tentativas automáticas quando a API pede para ir mais devagar, comum em contas novas; resposta cortada por limite de tokens gera erro claro. O parâmetro `temperature` foi removido porque a versão atual do SDK passou a recusá-lo.

8.3 Modo detalhado e modo rápido

	Detalhado (padrão)	Rápido (opcional)
Chamadas	1 por lote + 1 de síntese	1
Quem aponta as menções	O modelo, review a review, frase a frase	O modelo, listando as reviews de cada dimensão de uma vez
Contagens e confiança	Código	Código (a partir das listas)
Resultado com o Qwen 3B	98% de menções válidas, notas coerentes	Pulou a maioria das reviews ou não achou nenhuma

O modo rápido nasceu de uma pergunta natural: por que não mandar tudo numa chamada só? O tempo de um modelo local acompanha o tamanho da resposta, não o número de chamadas, e uma resposta só com as listas por dimensão é menor. O risco era o modelo não dar a mesma atenção a 40 reviews lidas de uma vez, e o teste confirmou (seção 11). Por isso o padrão é o detalhado; o rápido fica para modelos maiores. Se o rápido não encontrar nenhuma menção, o app refaz sozinho no detalhado.

O cache separa as análises por modo, e o script de validação consegue rodar os dois modos com os mesmos rótulos humanos, para medir a diferença.

8.4 Modo leitura

Sem `ANTHROPIC_API_KEY`, o app não cai num modo de demonstração com outra lógica: ele mostra as análises já salvas e continua coletando reviews, só sem gerar análises novas. É assim que alguém avalia o projeto sem chave (seção 9.4).

9. Aplicação e operação

O app foi desenhado para que o usuário quase nunca espere: a análise salva abre na hora, o analista adianta as dos perfumes populares, e o que falta é coletado e analisado sob demanda, com o andamento na tela.

9.1 Aplicação Streamlit

- **Duas formas de escolher o perfume:** buscar no catálogo (filtros por marca, gênero, nome e mínimo de reviews) ou colar o link de qualquer perfume do Fragrantica, em qualquer idioma do site. Colar o link foi a resposta ao requisito de o usuário poder analisar qualquer produto, e não só os do catálogo.

- **Cabeçalho:** nota e votos do site, reviews no banco, data da review mais recente, acordes e pirâmide olfativa.
- **Aba de análise:** resumo, gráfico das notas por dimensão com a cor da confiança, perfil de cliente ideal, pontos fortes e fracos e, em cada dimensão, o resumo, as contagens, o índice e as frases de evidência. O rodapé diz que o conteúdo foi gerado por IA, com modelo, data, nº de reviews e % de menções válidas.
- **Aba de reviews:** tabela pesquisável com as reviews do perfume, para conferir as evidências.
- **Aba Pipeline:** totais do banco, reviews novas por dia, fila do analista e análises por dia. Mostra que o pipeline está vivo.
- **Qual análise mostrar:** a que corresponde ao conjunto atual de reviews, preferindo a do analista (mais reviews), depois a detalhada do app e por último a rápida. Se chegaram reviews novas, mostra a última com um aviso e oferece "Gerar novamente".

9.2 Analista automático

Serviço que gera análises sozinho, em segundo plano. Foi criado quando a análise com o modelo local levava 16 minutos: em vez de o usuário esperar, os perfumes populares ficam prontos antes.

- **Fila calculada no banco:** perfumes reais com reviews informativas suficientes e sem análise do modelo atual que cubra a review mais recente; mais reviews primeiro. Sem estado próprio: reviews novas recolocam o perfume na fila.
- **Cede a vez ao app:** antes de cada chamada ao modelo, o analista confere a tabela `uso_llm`. Se o app estiver gerando uma análise pedida por alguém, ele espera. A GPU local atende uma chamada por vez, e quem está na tela tem prioridade. O usuário espera no máximo o trecho em andamento, nunca uma análise inteira. Marcas com mais de 1 hora são ignoradas, e o analista apaga as próprias ao iniciar, para um processo interrompido não deixar o app esperando.
- **Uma tabela, e não um tópico do Kafka, para isso:** "o app está usando o modelo agora?" é um estado, não um evento.
- **Proteções:** janela de horário opcional (pode virar a noite), até 50 análises por dia, 24 h fora da fila para quem falhou. Com o Claude, fica desligado a menos que seja autorizado (`ANALISTA_PERMITIR_PAGO`), para não gastar crédito sozinho.
- Lê até 100 reviews por perfume (`ANALISTA_MAX_REVIEWS`), nunca menos que o app: sem pressa, mais relatos e mais confiança.

9.3 Carga em lote

Um comando para popular o banco a partir de uma lista de links, por exemplo os perfumes das marcas mais populares em ordem de ranking. A seleção percorre o CSV na ordem e pega 2 perfumes de cada marca até completar 80. A coleta pula quem foi coletado há pouco, então pode ser repetida para tentar só os que falharam. A análise mostra a estimativa de custo e só gera com confirmação explícita (`SIM=1`); perfumes já analisados não são cobrados de novo.

Na decisão final, o catálogo cresce pelo uso: cada link colado vira um perfume coletado e analisado, disponível na hora para o próximo usuário. A carga em lote ficou como opção.

9.4 Avaliação sem chave: cópia dos dados

O banco fica num volume do Docker, fora do repositório. Para quem baixar o projeto ver os resultados sem chave de API e sem coletar nada:

- `make exportar` gera `data/snapshot/fragrantica.sql.gz` com perfumes, reviews e análises (dump só de dados, com `INSERT ... ON CONFLICT DO NOTHING`).
- `make restaurar` carrega o arquivo. Pode rodar mais de uma vez sem duplicar, e ajusta a sequência de ids das análises.
- Sem chave, o app abre em modo leitura (8.4).

Os dados fictícios de demonstração saem antes, com `make remover-exemplo` . A cópia leva o texto das reviews, com autores pseudonimizados; por isso o repositório deve ser privado.

9.5 Configuração

Toda configuração vem de variáveis de ambiente (`.env`), lidas num único objeto `Config` , com valores padrão funcionais. Os principais parâmetros e os valores atuais estão no README; os que mudam o comportamento da análise são `LLM_PROVEDOR` , `LLM_MAX_REVIEWS` (100), `MODO_ANALISE_APP` (detalhado), `FILTRAR_FRASES` (verdadeiro) e `MIN_REVIEWS_ANALISE` (5).

10. Qualidade: testes e validação

O repositório tem 72 testes automatizados (pytest), e todos passam. A qualidade da IA tem um protocolo próprio de validação contra rótulos humanos, já implementado; a rotulagem da amostra ainda precisa ser feita.

10.1 Testes automatizados

Arquivo	Testes	O que cobre
`test_analise.py`	19	Divisão em frases e filtro; seleção de reviews; índice suavizado e nota ancorada; descarte de review ou frase inexistente; agregação sem repetição; os dois modos de ponta a ponta; trava contra texto sem menções; esquemas sem referências para o *tool use*; ausência de frases copiáveis no prompt
`test_cliente_ollama.py`	8	Chamada ao Ollama contra um servidor HTTP falso: esquema, contexto, nova tentativa, modelo não baixado, Ollama fora do ar, tempo limite, modelos por modo
`test_analista.py`	7	Janela de horário (inclusive virando a noite), trava para provedor pago, espera enquanto o app usa o modelo, pausa de 24 h para falhas
`test_sob_demanda.py`	6	Leitura de links de qualquer idioma, decisão de coletar, pedido, atendimento no coletor e detecção de que as reviews chegaram
`test_cliente_anthropic.py`	5	Com um SDK falso: ferramenta forçada, validação, nova tentativa, piso de tokens, ausência de `temperature`
`test_etl.py`	5	Limpeza, datas relativas e absolutas, pseudonimização, hash, regra de review informativa
`test_pipeline.py`	5	Producer → barramento → consumer com um banco falso: reentrega sem duplicar, mensagens fora de ordem, DLQ
`test_scraper.py`	5	Interpretação de catálogo, ficha e reviews a partir do que o JavaScript devolve
`test_lote.py`	4	Leitura de CSV (qualquer separador, coluna de links em qualquer posição), seleção por marca, estimativa de custo
`test_models.py`	3	Contratos das mensagens

Arquivo	Testes	O que cobre
`test_integracao_postgres.py`	3	Contra um PostgreSQL 16 real: fluxo completo, cache, pedidos com status fora de ordem, fila do analista, marcas de uso, remoção dos dados fictícios
`test_validacao.py`	2	Métricas do script de validação

Decisões de teste:

- **Nenhum teste depende de rede, de chave de API ou do site.** O LLM é substituído pelo provedor `demo` ou por servidores e SDKs falsos; o Kafka, por um publicador em memória. Os testes rodam em segundos em qualquer máquina.
- **Os testes de integração usam um PostgreSQL de verdade** (pulados se `DATABASE_URL_TESTE` não estiver definida), porque idempotência, chaves estrangeiras e a consulta da fila só se provam no banco real.
- **Cada falha encontrada nos testes manuais virou um teste:** o trecho copiado das instruções, a resposta só com texto do modo rápido, o `temperature` recusado, a marca de uso que sobrou de um container interrompido.

O que os testes não cobrem: o Kafka real, o site ao vivo e as respostas reais dos modelos. Esses três foram verificados nos testes manuais da seção 11.

10.2 Protocolo de validação da IA

O script `scripts/validacao.py` compara a classificação do modelo com a de uma pessoa, review a review:

1. **Amostra:** sorteia 100 reviews informativas de um perfume (semente fixa) e grava uma planilha com as colunas `humano_<dimensão>`, vazias, e `llm_<dimensão>`, preenchidas pelo modelo.
2. **Rotulagem cega:** a pessoa preenche as colunas humanas com positiva, negativa ou neutra (ou deixa vazio) sem olhar as do modelo.
3. **Métricas por dimensão:** precisão e recall da detecção (a review fala ou não daquela dimensão), acurácia da polaridade quando os dois detectaram, e o kappa de Cohen em 4 classes (vazio, positiva, negativa, neutra), que desconta a concordância por acaso.
4. **Comparar configurações com os mesmos rótulos:** o comando `reclassificar` refaz só as colunas do modelo em outro modo ou com outro provedor. Assim dá para medir modo rápido contra detalhado, ou Qwen contra Claude, sem rotular de novo.

Leitura esperada das métricas: precisão baixa indica menções inventadas; recall baixo, reviews puladas (o problema previsto para o modo rápido); kappa acima de 0,6 costuma ser lido como concordância substancial.

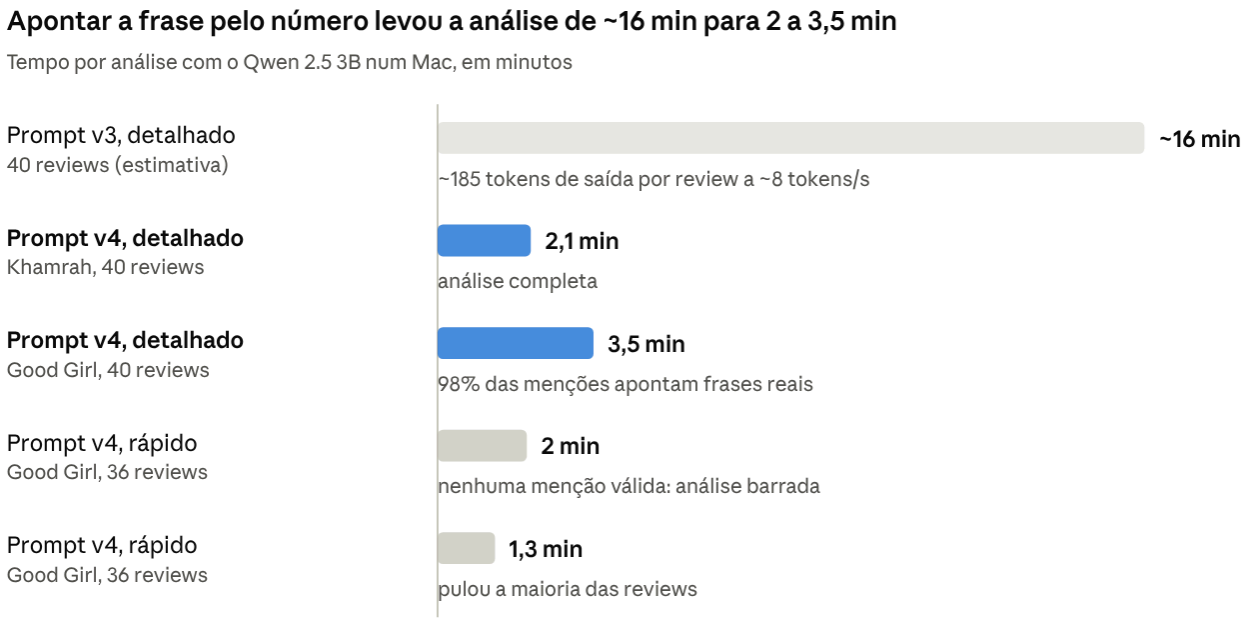
11. Resultados experimentais

Todos os números desta seção vêm de testes reais no Mac da autora, em 30/10/2026, com o pipeline completo (Kafka, coletor e banco). Por enquanto, eles são observações de poucos perfumes, e não uma avaliação estatística; a validação contra rótulos humanos (10.2) é o próximo passo.

11.1 Pipeline de ponta a ponta

- A carga de demonstração passou pelo Kafka e o consumer gravou 36 reviews; repetida, gravou 0 novas e 36 duplicadas, confirmando a idempotência.
- A coleta sob demanda pelo link funcionou para o Wind Flowers (Creed, 15 reviews), o Good Girl (Carolina Herrera, 209) e o Khamrah (Lattafa, 207), passando pelo Cloudflare.
- O analista automático analisou sozinho o Khamrah e o Wind Flowers logo ao subir, e cedeu a vez ao app quando uma análise foi pedida na tela.

11.2 Tempo



medido no app (duracao_s na tabela analises) · Qwen 2.5 3B num Mac · 30/10/2026; v3 é estimativa

O tempo de um modelo local acompanha o número de tokens que ele escreve, a cerca de 8 a 11 tokens por segundo neste Mac. Reduzir a resposta (só as menções existentes, chaves de uma letra, o número da frase no lugar do trecho) foi o que trouxe o ganho, sem trocar

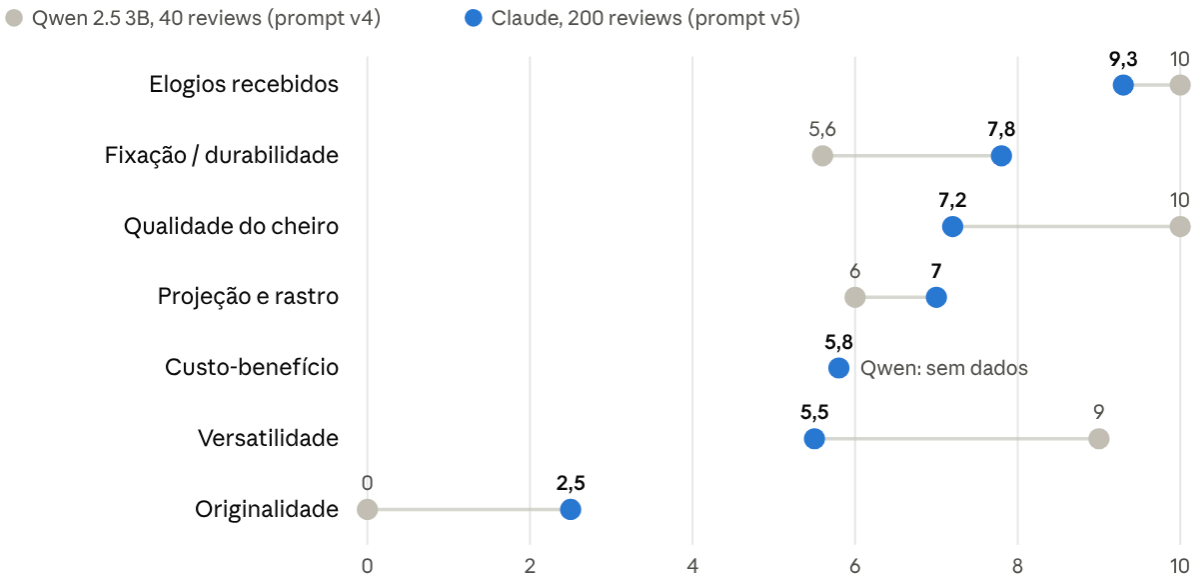
de modelo. O modo rápido economizou 1 a 2 minutos a mais e falhou nas duas tentativas.

11.3 Qualidade

Configuração	Perfume e reviews	Menções válidas	Confiança	Texto
Qwen 3B, prompt v1	Wind Flowers, 15	nenhuma extraída	—	Análise vazia
Qwen 3B, prompt v2	Wind Flowers, 15	4 de 5 evidências de fixação inventadas	"alta" em fixação, com contagem inflada	Nota vazia, listas vazias, "afixação"
Qwen 3B, prompt v4, rápido	Good Girl, 36	1ª tentativa: nenhuma; 2ª: poucas	baixa em quase todas	Notas genéricas (7 e 5), contradições
Qwen 3B, prompt v4, detalhado	Good Girl, 40	98%	média em 5 dimensões	Correto, mas pobre ("cheirinho"), "indicado para" vazio
Claude, prompt v5, detalhado	Good Girl, 200	100%	alta nas 7	Claro e coerente; identificou que o perfume é visto como "batido" por causa dos *dupes*

O Qwen com 40 reviews dá notas extremas (10 e 0); o Claude, com 200, fica entre 2,5 e 9,3

Good Girl (Carolina Herrera), nota de 0 a 10 por dimensão; Qwen antes da suavização do índice



análises do Good Girl salvas no banco · 30/10/2026

As notas do Claude concordam com a reputação do Good Girl: muitos elogios de terceiros, boa fixação e originalidade baixa pela quantidade de perfumes parecidos. O Qwen acertou a direção em quase tudo, mas com notas extremas, apoiadas em poucos relatos; foi o que motivou a suavização do índice e a nota ancorada nele (v5). A maior divergência é a versatilidade (9,0 contra 5,5): com 200 reviews, o Claude encontrou muitos relatos de que o perfume funciona melhor à noite e no frio.

11.4 Custo

As primeiras análises de teste com o Claude, com 200 reviews cada, custaram US$ 0,20 no total. Com 100 reviews (o padrão atual), a estimativa é de US$ 0,05 a 0,10 por análise. Como cada análise fica no banco e só é refeita quando chegam reviews novas, o custo acompanha os perfumes analisados, não o número de visitas.

12. Considerações finais

O projeto entrega o fluxo pedido de ponta a ponta: coleta incremental, streaming pelo Kafka, banco, aplicação e análise por IA com notas por dimensão e perfil de cliente ideal. O resultado final é rastreável: cada nota tem uma contagem por trás e cada afirmação, uma frase real de review. O que mais pesou na qualidade não foi a escolha do modelo, e sim o desenho de como o modelo é usado.

12.1 Principais achados

1. **Dividir o trabalho entre código e modelo foi a decisão mais importante.** O modelo aponta e escreve; o código conta, calcula o índice e a confiança e confere as evidências. Com isso, os números da análise são verificáveis e as evidências não podem ser inventadas, mesmo com um modelo de 3B.
2. **Com modelos pequenos, o formato da resposta pesa mais que a redação das instruções.** Listas opcionais voltaram vazias; opções fechadas por dimensão funcionaram. E tudo o que pode ser copiado das instruções é copiado: a frase "dura o dia todo" saiu dos exemplos e virou evidência falsa.
3. **Num modelo local, o tempo acompanha os tokens de saída.** Apontar a frase pelo número, em vez de copiá-la, e listar só as menções existentes reduziu a análise de ~16 minutos para 2 a 3,5, com 98% de menções válidas, sem trocar de modelo.
4. **Ler tudo numa chamada só não compensou.** O modo rápido ganhou 1 a 2 minutos e perdeu a maior parte das reviews. A extração estruturada, review a review, foi superior ao resumo direto, que é justamente o que um uso ingênuo de LLM faria.
5. **Confiança depende de volume.** Com 40 reviews, quase nenhuma dimensão chegou aos 15 relatos da confiança alta; com 200, todas chegaram. Poucos relatos também geram notas extremas, o que justificou a suavização do índice.
6. **O modelo local resolve a estrutura, mas não o texto.** O Qwen 3B seguiu o formato, mas escreveu mal em português; o 7B não coube junto com o Docker num Mac comum. A API paga resolveu a qualidade a centavos por análise, e o cache impede que o custo cresça com o número de visitas.
7. **Os problemas de streaming eram reais.** Review chegando antes da ficha e status fora de ordem apareceram nos primeiros testes. A combinação de entrega pelo menos uma vez com gravação idempotente resolveu os dois sem lógica especial no produtor.
8. **A espera do usuário é um problema de arquitetura, não só de modelo.** Cache, analista em segundo plano que cede a vez e coleta sob demanda limitada às reviews que a análise usa fizeram a espera acontecer só no primeiro pedido de cada perfume.

12.2 Limitações

Limitação	Efeito
A validação contra rótulos humanos ainda não foi executada	A qualidade da extração foi observada em poucos perfumes, sem precisão, recall e kappa medidos
A coleta depende do HTML do Fragrantica e do Cloudflare	Uma mudança no site ou no bloqueio para a coleta; os seletores ficam num arquivo só para facilitar o ajuste

Limitação	Efeito
O filtro de frases usa palavras-chave em português e inglês	Uma frase relevante sem nenhuma pista pode não chegar ao modelo (risco de recall), e reviews em outros idiomas são pouco filtradas
A confiança conta relatos, mas não mede a concordância entre eles	15 relatos divididos meio a meio e 15 unânimes recebem a mesma confiança
As reviews refletem quem escreve reviews	Viés de seleção: entusiastas e insatisfeitos falam mais; reformulações do perfume misturam versões
Notas de modelos diferentes não são comparáveis	O cache separa as análises por modelo, mas uma nota 7 do Qwen e uma 7 do Claude não significam o mesmo
Infraestrutura de uma máquina	Kafka com um broker e sem réplicas; app sem autenticação; volume não testado em escala
Custos são estimativas	O valor real por análise depende do tamanho das reviews e fica no painel da Anthropic

12.3 Melhorias futuras

- [ ] **Executar a validação humana:** 100 reviews de 2 a 3 perfumes, comparando Claude e Qwen e os modos detalhado e rápido com os mesmos rótulos.
- [ ] **Base de conhecimento por família olfativa e acordes:** guardar as menções por review numa tabela e agregá-las por acorde, para (a) estimar notas de perfumes sem reviews a partir dos parecidos, sempre separadas e rotuladas como estimativa; (b) dar contexto à síntese ("fixa acima da média dos florais"); (c) recomendar perfumes parecidos. Exige testar antes se as notas de fixação e projeção se repetem entre perfumes do mesmo grupo, porque dependem mais da concentração que das notas.
- [ ] **Confiança que considere a concordância** (por exemplo, um intervalo de confiança para a proporção de elogios), e não só o número de relatos.
- [ ] **Evolução no tempo:** notas por período, para detectar reformulações ("a fixação caiu nas reviews de 2025").
- [ ] **Custo menor na API:** cache do prompt de sistema, que se repete em todo lote, e o processamento em lotes assíncronos da API para o analista, que não tem pressa.
- [ ] **Provedor compatível com a API da OpenAI**, para usar modelos abertos maiores hospedados (Groq, Together, OpenRouter) sem pesar na máquina.

- [ ] **Coletar a família olfativa** que o site informa, além dos acordes.
- [ ] **Monitoramento da coleta:** alerta quando um perfume popular volta com 0 reviews, sinal de que o seletor do site mudou.
- [ ] **Publicação numa VM** com HTTPS e autenticação no app, e Kafka com mais de um broker se o volume crescer.

Referências

- CHERNEV, A.; BÖCKENHOLT, U.; GOODMAN, J. When product assortment leads to choice overload: a conceptual review and meta-analysis. *Journal of Consumer Psychology*, n. 2, p. 333–358, 2015. Kellogg School of Management.
- CHEVALIER, J. A.; MAYZLIN, D. The effect of word of mouth on sales: online book reviews. *NBER Working Paper* n. 10148. NBER Digest, maio de 2004.
- HU, M.; LIU, B. Mining and summarizing customer reviews. In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2004. dblp; resumo do método.
- IYENGAR, S. S.; LEPPER, M. R. When choice is demotivating: can one desire too much of a good thing? *Journal of Personality and Social Psychology*, v. 79, n. 6, p. 995–1006, 2000. Artigo (PDF).
- PONTIKI, M. et al. SemEval-2014 Task 4: aspect based sentiment analysis. In: *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, p. 27–35. Association for Computational Linguistics, 2014. ACL Anthology.
- SCHWARTZ, B. *The paradox of choice: why more is less*. Nova York: Ecco, 2004. Publishers Weekly.